

Problem 2

Please note that the quality of the model is not at its best because I was not able to run it on GPU, because of a problem with resource allocation on the servers. While others might have had access to the GPU, when I ran the code, it seems that I didn't get a slot. For this reason, I couldn't run my model long enough.

Instructions for the Code

The following steps are the steps of implementing the code:

1. First we import all the needed packages such as gym, gym, numpy, matplotlib.pyplot, tensorflow, keras, tensorflow_probability, warnings, & math.
2. We define the function to process the image:

```
def preprocess(image):  
    """Preprocess a frame from the game."""  
    image = image[35:195] # Crop  
    image = image[:, :2, 0] # Downsample by a factor of 2  
    image[image == 144] = 0 # Erase background (background type 1)  
    image[image == 109] = 0 # Erase background (background type 2)  
    image[image != 0] = 1 # Set paddles and ball to 1, everything else to 0  
    return np.reshape(image.astype(np.float32).ravel(), [80, 80])
```

3. Next, we define the environment using the code below such that it allows us to simulate the game Pong-V0, version 0. This environment is a built-simulated environment using gym. We also define some parameters for the neural network and policy gradient.

```
# hyperparameters  
learning_rate = 0.0001  
gamma = 0.99  
#num_episodes = 1500  
render = False  
#tf.get_logger().setLevel(tf.compat.v1.Logging.ERROR)  
#import tensorflow.compat.v1 as tf  
#tf.disable_eager_execution()  
#tf.logging.set_verbosity('ERROR')  
  
env = gym.make("Pong-v0")
```

4. Next, we build a simple neural network. Since the game doesn't include many variables; the model below is sufficient. The model includes one input layer, one layer to flatten the images, and one fully connected layer with 256 neurons and a linear activation function. The output layer has an activation function: SoftMax.

```

model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(80, 80, 2)))
#model.add(tf.keras.layers.Conv2D(2, (80,80), activation='relu'))
#model.add(tf.keras.layers.BatchNormalization())
#model.add(tf.keras.layers.MaxPooling2D((1, 1)))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(256, activation='linear'))
#model.add(tf.keras.layers.Dense(256, activation='linear'))
model.add(tf.keras.layers.Dense(2, activation='softmax'))

optimizer = tf.keras.optimizers.Adam(learning_rate)

```

Linear activation function: It has the formula $f(z) = z$, for each input vector z .

Softmax activation function: is a mathematical function that is often used to convert a vector of raw scores or logits into a probability distribution, where the goal is to assign an input to one of several possible classes. For each element i in the input vector z :

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

This neural network is used to train the trainable values (θ , of the policy that is used to play the game, meaning decide between the two actions of the game, left or right.

5. Next an optimizer is defined using keras. Adam is selected.
6. The discount factor $\gamma = 0.99$ is defined.
7. Lists are initiated outside of the training loop to:
 - track the rewards of the last 100 episodes
 - Track all the rewards of all the episodes
8. A for loop is started that runs over batches of episodes. At each batch, a certain number of episodes is sampled, in this case (5). Next, empty lists are initiated to keep track of actions, rewards, observations (these weren't used afterwards in the code). And at the beginning of the batch, the loss is initiated to 0. Afterwards `tf.gradienttape` is used to keep track of the functions and the model, to be able to compute the gradient afterwards:

The `tf.GradientTape()` is a context manager provided by TensorFlow that is used for automatic differentiation, specifically for computing gradients. Context Management: When you enter a block of code using with `tf.GradientTape()` as `tape:`, you are creating a context where TensorFlow will "record" the operations that occur inside this block. It's like turning on a tape recorder for TensorFlow to keep track of what's happening.

TensorFlow uses this recorded information to automatically compute gradients of operations with respect to their inputs.

With each loop across the batch, another loop is created to loop over episodes.

```
for batch in range(0, 200):
    num_episodes = 5
    # Training Loop
    obs_history_batch = []
    reward_history_batch = []
    action_history_batch = []
    discounted_rewards_batch = []

    loss = 0
    with tf.GradientTape() as tape:
        #tape.watch(model.trainable_variables)
```

8.1. Within each loop of the episode, we initiate the following lists and variables:

```
for episode in range(num_episodes):
    obs_history_eps = []
    reward_history_eps = []
    action_history_eps = []
    selected_probs_eps = []

    index_history_eps = []
    prob_output_eps = tf.Variable(tf.zeros([0, 2], dtype=tf.float32))

    state = preprocess(env.reset()[0])
    prev_state = state
    episode_reward = 0
    #reward_history = []
    action_probs = []
    #selected_probs_eps = []

    terminated = False
    truncated = False

    #with tf.GradientTape() as tape:

        while (not terminated) and (not truncated):
```

The variables terminated and truncated help us track when to stop the episode based on the output of each step of the environment. They are initiated to false. The other lists

track the states, rewards of an episode, actions, and the probability that is outputted by the neural network for the action that ends up being selected.

Afterwards, the loop where the episode takes place is initiated.

8.1.1. The loop of the episode is created, such that it ends when the episode is terminated or truncated.

8.1.2. The first step is to sample an action from the policy. So, we use the current step in addition to the previous step to give directionality of the action, as an input to the model (the neural network).

```
while (not terminated) and (not truncated):

    # Predict the probability of each action
    with open('output.log', 'w') as log_file:
        with contextlib.redirect_stdout(log_file), contextlib.redirect_stderr(log_file):
            if episode_reward==0:
                combined_states = tf.stack([tf.convert_to_tensor(prev_state), tf.convert_to_tensor(prev_
                action_prob = model(np.expand_dims(combined_states, axis=0))[0]
            else:
                action_prob = model(np.expand_dims(combined_states, axis=0))[0]

    #print(action_prob)
    ##normalize the probabilities because they don't sum to 1
    #action_prob /= np.sum(action_prob)
    #action = np.random.choice([2, 3], p=action_prob)
    #action = np.where(np.random.multinomial(1, action_prob) == 1)[0][0] + 2
    dist = tfp.distributions.Categorical(probs=action_prob, dtype=tf.float32)
    action = dist.sample()
    #print(int(action))
    if int(action)==0:
        a=2
    elif int(action)==1:
        a=3
```

8.1.3. The observation is converted into a tensor and inputted into the model; the output of the corresponding model is the policy π_{θ} . Note that the θ variables are hidden in this case in the neural network. The output of the model represents a probability vector.

8.1.4. In order to sample an action, we feed the probability function into `tfp.distributions.Categorical` in tensorflow probability, this function creates a distribution based on the two probabilities. Then we use this distribution to sample an action (2 or 3) that represents left or right.

8.1.5. Using the action we sampled, the agent takes the next action, and we collect the observation (to be used in the next state to decide on a policy), the reward, action, and selected probability of the action that was selected.

8.2. Outside of the while loop, once the episode ends, the sum of the rewards of the episode is tracked.

```
prev_state = next_state
    #if terminated or truncated:
        #episode_rewards.append(episode_reward)
episode_rewards.append(episode_reward)
print(f"Episode: {episode+batch*num_episodes}, Reward: {episode_reward}")
if len(episode_rewards) >= 100:
    avg_reward = np.mean(episode_rewards[-100:])
    print(f"Average Reward (Last 100 Episodes): {avg_reward}")
    #break
ter the end of the episode
discounted_rewards_eps = []
reward_sum = 0
for r in reward_history_eps[::-1]:
    reward_sum = r + gamma * reward_sum
    discounted_rewards_eps.insert(0, reward_sum)

#predicted_action_prob = model(np.expand_dims(combined_states, axis=0)) #shape of two
#loss = -tf.reduce_sum(tf.math.log(predicted_action_prob[0][a]) *discounted_rewards)
discounted_rewards_eps = np.array(discounted_rewards_eps)
reshaped_discounted_rewards = discounted_rewards_eps.reshape(-1, 1)
#tensor_logprob = tf.math.log(tf.convert_to_t(selected_probs_eps, dtype=tf.float32))
```

8.3. Then the discounted rewards of that episode is computed in a vector such that, we loop in the reverse order throughout the rewards of the episodes, and compute the discounted reward of the last state. Then for the state before the last, we add the reward sum of the last and add the discounted reward of the corresponding state, and so on. This vector will include the discounted reward for each state, where it include the rewards to go from that state.

8.4. The discounted rewards vector is converted to a NumPy array with shape (T, 1), where T is the number of states in an episode.

8.5. Next, we convert the discounted rewards to a tensor, and we convert the vector of selected probabilities into a tensor.

8.6. We get the log of the probabilities.

8.7. We add the loss computed for this episode to the one computed for previous episodes in the same batch.

```

index_history_eps = tf.cast(index_history_eps, dtype=tf.int32) # Or tf.int64, depending on your needs

selected_values = tf.math.log(tf.gather(prob_output_eps, index_history_eps, axis=1))

tensor_logprob = tf.math.log(tf.convert_to_tensor(selected_probs_eps, dtype=tf.float32))
tensor_rewards = tf.convert_to_tensor(reshaped_discounted_rewards, dtype=tf.float32)
###normalize the discounted rewards
mean = tf.reduce_mean(tensor_rewards)
std = tf.math.reduce_std(tensor_rewards)

# Normalize the discounted rewards
normalized_rewards = (tensor_rewards - mean) / (std + 1e-8)
tensor_rewards_normalized = tf.convert_to_tensor(normalized_rewards, dtype=tf.float32)

#loss = loss + tf.reduce_sum (index_history_eps * tensor_rewards_normalized)
for idx in range(0, tensor_rewards_normalized.shape[0]):
    loss = loss + tensor_rewards_normalized[idx] * tf.math.log(prob_output_eps[idx][int(index_history_eps[

```

8.8. For each batch, we convert the loss of the batch to negative, as the default process is to minimize the loss, and in this case, we want to maximize the loss which represents the reward).

8.9. We get the gradient of the of the policy

```

#LOSS = -LOSS
loss = -tf.convert_to_tensor(loss)
grads = tape.gradient(loss, model.trainable_variables)

```

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t) R[t] \right]$$

8.10. Then we apply the gradient using the optimizer:

```
optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

9. We create plots, print values of rewards every 100 episodes, and close the environment.

Results

The following plot shows the trend of the rewards of the episodes as the number of episodes increases. It also shows the moving average across the consecutive 100 episodes.

The model shows a learning trend of the agent; when tracking the moving average of the last 100 episodes. The model has a lot of noise and high variance as expected when using REINFORCE algorithm.

Note that the model was not tested for long periods due to the limitation of the availability of the server, as it crashed often. With enough trials and errors, and training for longer episodes the model can improve; same as the computational time.

The model can be improved by following one or more of the following steps:

- Using the difference of images of the previous state and the next state as an input instead of stacking the two frames.
- Changing the architecture of the neural network (note that several models were tested already)
- Changing the learning rate
- Changing the optimizer
- Include a variable learning rate
- Add regularization to the gradient descent
- Add dropout layer to avoid overfitting or getting stuck in a local minimum
- Add gradient clipping.

Acknowledgments:

For debugging the code, understanding syntax and role of functions, and learning about different implementation methods:

- TA
- Tensorflow tutorials
- ChatGPT
- Discussions with classmates
- Youtube
-